KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

클래스 4
캡슐화

칸은 안에 두고
문으로만 드나들기

```
class Secret:
    def __init__(self):
        self.__password = 1234

mypass = Secret()
print(mypass._Secret__password)
# 1234
```

오늘의 순서

1	왜 필요한가	아무 값이나 들어가는 도장 칸, 칸과 문 그림	4–5쪽
2	밑줄 하나 _	안에서만 쓰는 칸, 표지판, 밖에서도 바뀌어요	6–8쪽
3	밑줄 두 개 __	못 찾는 이름, 바뀐 이름, 우회해서 조회, 눌러 보기	9–13쪽
4	getter와 setter	꺼내는 문과 검사하는 문, 괄호 없이, 한 표 비교	14–18쪽
5	오류와 함정	밖에서 붙은 새 칸, 자식 틀, 괄호, setter 없음	19–22쪽
6	실습	열 개에서 멈추는 도장 카드, 키오스크 비밀번호	23–26쪽

3쪽은 노트북 준비, 27–28쪽은 막힐 때와 한 장 요약이에요.

들어가기

26에서 이어서 해요

✓	class, __init__, self, 메서드	24
✓	메서드로 상태 바꾸기와 if로 막기	25
✓	상속, super()	26
✓	if, else	17
✓	딕셔너리 {키: 값}	15

기억이 안 나면 왼쪽에 적힌 챕터를 먼저 다시 봐요.

노트북 다시 열기

1. VS Code로 hello-uv 폴더 열기
2. 탐색기에서 `basics.ipynb` 열기
3. 오른쪽 위 커널이 .venv인지 보기
4. 셀에 적고 `Shift` + `Enter` 로 실행

도장 수를 밖에서 아무 값으로나 바꿀 수 있어요

[1]
```
class Card:
    def __init__(self, name):
        self.name = name
        self.stamps = 0

    def add(self):
        self.stamps = self.stamps + 1

kim = Card("kim")
kim.add()
kim.stamps = 100
print(kim.name, kim.stamps)
```

← 도장은 add()로 한 개씩

← 직접 100을 넣었어요

```
kim 100
```

[2]
```
kim.stamps = -5
print(kim.name, kim.stamps)
```

← 음수도 들어가요

```
kim -5
```

kim Card

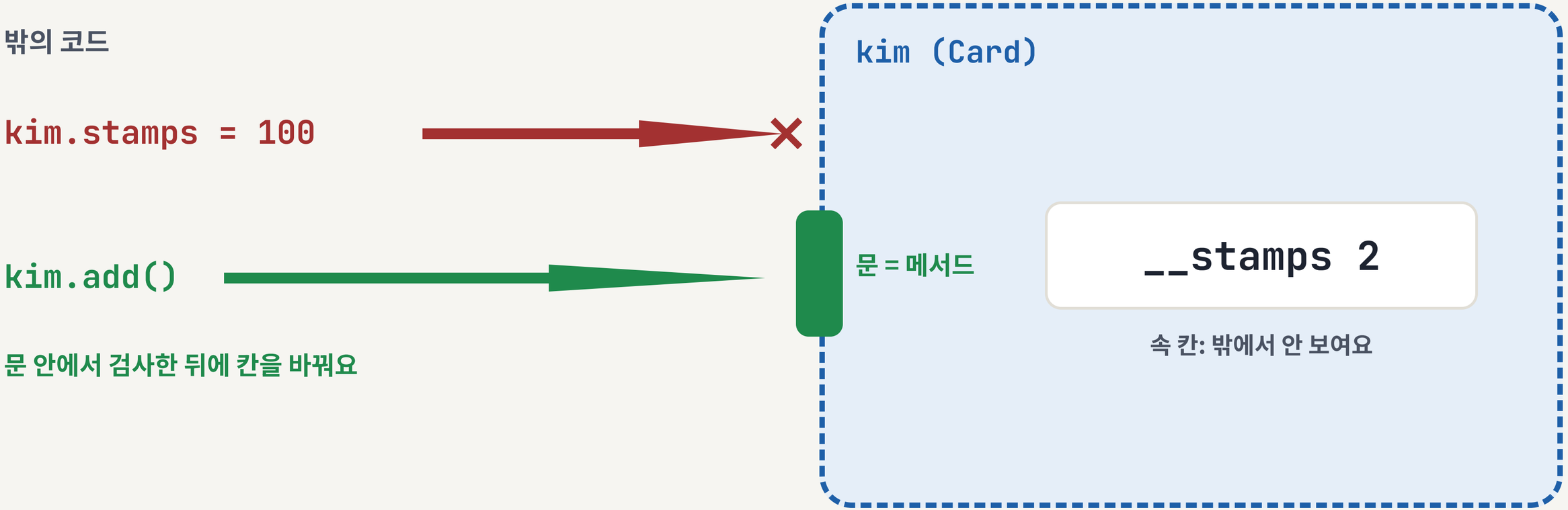
name	"kim"
stamps	-5

카드에는 오류도 경고도 없어요

도장 열 개를 모으지 않고도
다 채울 수 있어요

1. 왜 필요한가

칸은 안에 감추고, 메서드라는 문으로만 드나들어요



`_stamps`	2파트: 밑줄 하나, 손대지 말자는 약속
`__stamps`	3파트: 밑줄 두 개, 이름이 바뀌어 숨어요
`get_, set_, property`	4파트: 꺼내고 넣는 문 만들기

2. 밑줄 하나 _

밑줄 하나로 시작하는 칸은 안에서만 써요

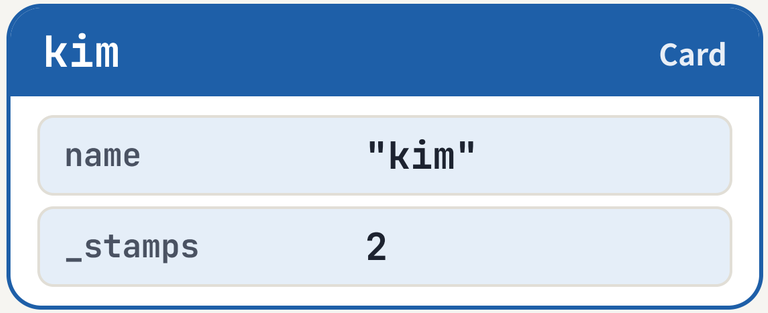
[1]
```
class Card:
    def __init__(self, name):
        self.name = name
        self._stamps = 0  ← 칸 이름 앞에 _ 하나

    def add(self):
        self._stamps = self._stamps + 1  ← 도장 찍는 문

    def count(self):
        return self._stamps  ← 도장 수를 꺼내 주는 문

kim = Card("kim")
kim.add()
kim.add()  ← 밖에서는 문만 써요
print(kim.count())
```

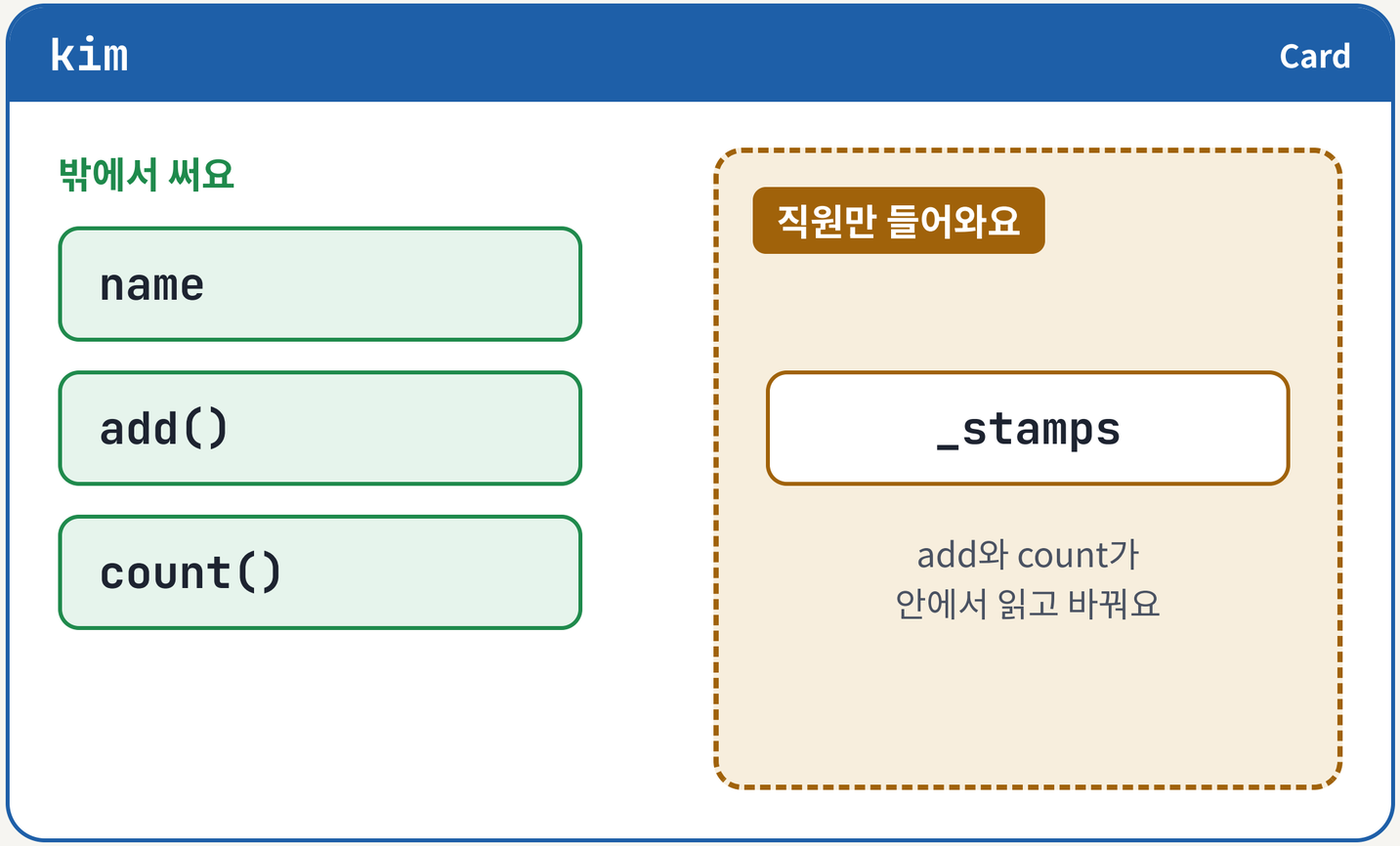
```
2
```



밖의 코드는 _stamps를 직접 적지 않았어요

2. 밑줄 하나 _

밑줄 하나는 '직원만 들어와요' 표지판이에요



표지판

_는 '안에서만 쓸게요'라는 표시예요

문은 안 잠겨요

밖에서 kim._stamps로 읽고 바꿀 수는 있어요

약속을 지켜요

밑줄 하나짜리 이름은 밖에서 안 써요

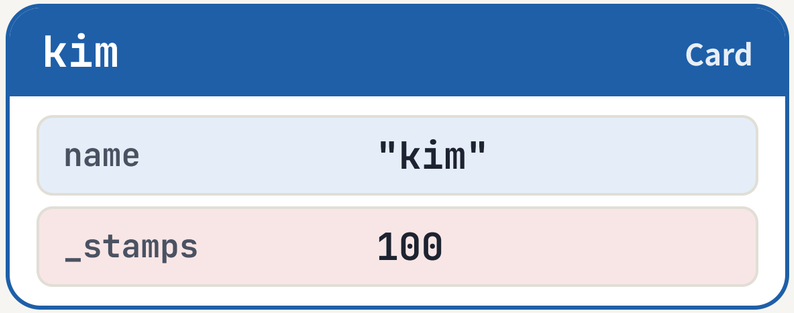
파이썬은 잠그지 않아요. 밑줄은 사람끼리 지키는 약속이에요.

2. 밑줄 하나 _

밖에서 _stamps에 넣어도 오류가 나지 않아요

```
[2] kim._stamps = 100     ← 표지판을 무시하고 들어갔어요
    print(kim.count())

    100                   ← add를 안 거치고 100이 됐어요
```



밖의 코드	읽는 사람에게는
`kim.stamps = 100`	1파트: 누구나 바꿔도 되는 칸처럼 보여요
`kim._stamps = 100`	되긴 되지만, 밑줄을 보고도 약속을 어긴 코드예요

밑줄 하나는 실수를 막아 주지, 일부러 하는 사람은 못 막아요.

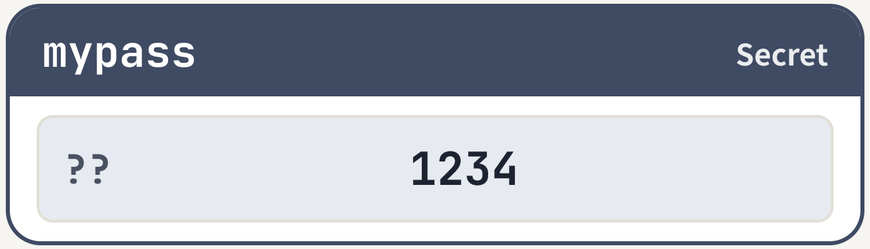
3. 밑줄 두 개 __

밑줄 두 개로 시작하는 칸은 밖에서 이름으로 못 찾아요

[1]
```
class Secret:
    def __init__(self):
        self.__password = 1234   ← 칸 이름 앞에 _ 두 개

mypass = Secret()
print(mypass.__password)
```

```
AttributeError: 'Secret' object has no attribute '__password'
```



칸은 분명히 있어요
그런데 이름이 __password가 아니에요

안에 1234가 있는데 왜 못 찾을까요? 두 장 뒤에서 봐요.

3. 밑줄 두 개 __

클래스 안의 메서드는 self.__password를 그대로 써요

[2]
```
class Secret:
    def __init__(self):
        self.__password = 1234

    def check(self, pw):
        return self.__password == pw

mypass = Secret()
print(mypass.check(1234))
print(mypass.check(1111))
```

← 안에서는 이름 그대로 읽혀요

```
True
False
```

← 1234가 맞아요

← 1111은 틀려요

밖에는 맞는지 틀린지만 알려 주고, 비밀번호는 보여 주지 않아요.

3. 밑줄 두 개 __

파이썬이 __password를 _Secret__password로 바꿔 둬요

```
[3] print(mypass.__dict__)   ← 객체 속 칸을 딕셔너리로 보여 줘요
```

```
{'_Secret__password': 1234}
```

클래스 안에 적은 이름

self._Secret__password

앞에 _클래스이름을 붙여서 저장해요

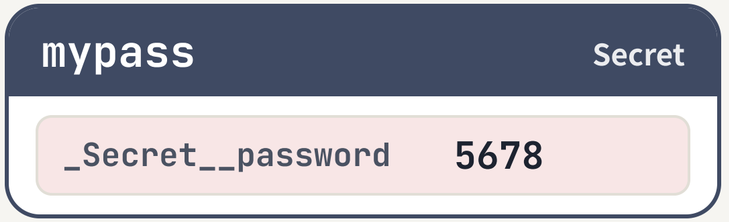
mypass.__password	그런 이름은 없어요	AttributeError
mypass._Secret__password	바뀐 이름 그대로 적으면	1234

3. 밑줄 두 개 __

바뀐 이름을 알면 밖에서도 꺼내고 바꿀 수 있어요

```
[4] print(mypass._Secret__password)   ← 바뀐 이름으로 돌아서 조회
1234

[5] mypass._Secret__password = 5678   ← 바꾸기까지 돼요
print(mypass.check(5678))
True
```



자물쇠가 아니에요	밑줄 두 개도 이름을 바꿔 둔 것뿐이에요
그래서 쓰는 곳	실수로 같은 이름을 쓰는 걸 막을 때 (5파트)
우회는 하지 않아요	_Secret__password를 밖에서 쓰는 코드는 만들지 않아요

3. 밑줄 두 개 __

눌러 보기: 어떤 이름으로 저장되고, 어떤 이름으로 찾힐까요

① 클래스 안에 적을 이름

- `self.password`
- `self._password`
- `self.__password`

② 밖에서 꺼낼 이름

- `mypass.password`
- `mypass._password`
- `mypass.__password`
- `mypass._Secret__password`

③ 실행 ▶

```
class Secret:
    def __init__(self):
        self.__password = 1234

mypass = Secret()
print(mypass.__password)
```

저장된 이름 (mypass.__dict__)

실행을 누르면 나와요

출력

실행을 누르면 나와요

안에 적을 이름과 밖에서 꺼낼 이름을 고르고 실행을 눌러요. 브라우저로 슬라이드를 열면 눌러져요.

4. getter와 setter

꺼내는 문은 getter, 넣는 문은 setter예요

[1]
```
class Secret:
    def __init__(self):
        self.__password = 1234

    def get_password(self):
        return self.__password

    def set_password(self, new):
        if 1000 <= new <= 9999:
            self.__password = new
        else:
            print("비밀번호는 네 자리 숫자예요")

mypass = Secret()
print(mypass.get_password())
```

getter: 값을 꺼내 돌려줘요

setter: 검사하고 넣어요

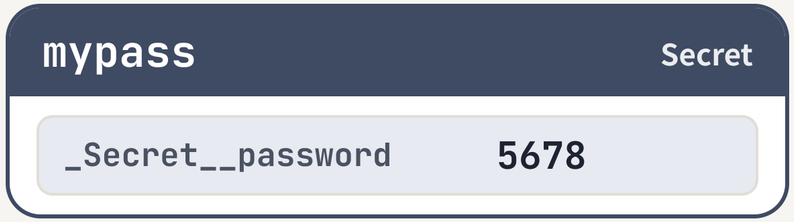
밖에서는 문으로

```
1234
```

setter가 검사해서 틀린 값은 안 넣어요

[2]
```
mypass.set_password(12)          ← 12는 네 자리가 아니에요
print(mypass.get_password())     ← 그대로 1234
mypass.set_password(5678)        ← 검사를 통과해요
print(mypass.get_password())     ← 5678로 바뀌었어요
```

```
비밀번호는 네 자리 숫자예요
1234
5678
```



검사 없이 칸을 그대로 열어 두면 (참고)

```
mypass = Secret()
mypass.password = 12     ← 12가 그대로 들어가요
print(mypass.password)
```

```
12
```

4. getter와 setter

@property를 붙이면 괄호 없이 꺼내요

[3]
```
class Secret:
    def __init__(self):
        self.__password = 1234

    @property                          ← def 바로 위에 붙이는 표시
    def password(self):
        return self.__password         ← getter와 같은 일, 이름은 칸 이름

mypass = Secret()
print(mypass.password)  ← 괄호 없이 칸처럼 읽어요
```

```
1234
```

꺼내는 코드가 이렇게 짧아져요

mypass.password

@password.setter를 붙이면 = 으로 넣어도 검사해요

[4]
```
class Secret:
    def __init__(self):
        self.__password = 1234

    @property
    def password(self):
        return self.__password

    @password.setter
    def password(self, new):
        if 1000 <= new <= 9999:
            self.__password = new
        else:
            print("비밀번호는 네 자리 숫자예요")

mypass = Secret()
mypass.password = 12
print(mypass.password)
mypass.password = 5678
print(mypass.password)
```

← set_password와 같은 검사

← = 으로 넣어도 setter가 불려요

← 통과

```
비밀번호는 네 자리 숫자예요
1234
5678
```

같은 일, 두 가지 모양

꺼내기	`get_password()`
	`password`
넣기	`set_password(5678)`
	`password = 5678`

둘 다 같은 일을 해요
밖에서 칸처럼 쓰고 싶으면 property를 골라요

4. getter와 setter

다섯 가지를 한 표로 비교해요

안에 적는 모양	밖에서	잠금	언제
`self.stamps`	읽기, 바꾸기	그대로 열림	아무나 바꿔도 될 때 (보통은 이것)
`self._stamps`	되긴 해요	약속	안에서만 쓰는 칸, 자식 틀도 쓸 칸
`self.__stamps`	이름으로는 안 돼요	이름이 바뀜	자식 틀과 이름이 겹치면 안 될 때
`get_, set_`	메서드로	문	꺼내고 넣을 때 검사가 필요할 때
`@property`	칸처럼	문	getter, setter를 칸처럼 쓰고 싶을 때

밑줄은 자물쇠가 아니에요. 밖에서 쓸 문을 정해 두는 게 캡슐화예요.

밖에서 mypass.__password = 0 이라고 적으면 새 칸이 생겨요

[1]
```
class Secret:
    def __init__(self):
        self.__password = 1234

    def check(self, pw):
        return self.__password == pw

mypass = Secret()
mypass.__password = 0   ← 비밀번호를 0으로 바꾼 줄 알았어요
print(mypass.check(1234))
print(mypass.__dict__)
```

```
True   ← 1234가 아직 맞아요
{'_Secret__password': 1234, '__password': 0}
```

진짜 비밀번호 칸 — 밖에서 새로 붙은 칸

이름 바꾸기는 클래스 안에서만 일어나요. 밖에서 적은 __password는 그냥 새 칸이에요.

5. 오류와 함정

자식 틀에서 부모의 __stamps를 쓰면 이름이 달라서 못 찾아요

[2]
```
class Card:
    def __init__(self, name):
        self.name = name
        self.__stamps = 0                          ← Card 안: _Card__stamps로 저장

    def add(self):
        self.__stamps = self.__stamps + 1

class VipCard(Card):
    def add(self):
        self.__stamps = self.__stamps + 2  ← VipCard 안: _VipCard__stamps를 찾아요

lee = VipCard("lee")
lee.add()
```

```
AttributeError: 'VipCard' object has no attribute '_VipCard__stamps'
```

부모와 자식이 같이 쓰는 칸은 밑줄 하나로

✓ `self._stamps`

고친 뒤 lee.add() 하면 lee._stamps는 2 (실행 결과)

property에 괄호를 붙이면 'int' object is not callable

[3]
```
class Card:
    def __init__(self, name):
        self.name = name
        self.__stamps = 0

    @property
    def stamps(self):
        return self.__stamps

kim = Card("kim")
print(kim.stamps())   ← 메서드처럼 괄호를 붙였어요
```

```
TypeError: 'int' object is not callable
```
← kim.stamps가 벌써 0이라서 0()이 돼요

× ~~print(kim.stamps())~~ → ✓ `print(kim.stamps)`

5. 오류와 함정

setter가 없는 property에 넣으면 has no setter

```
[4]  kim.stamps = 100   ← 앞 장의 kim, getter만 있어요

AttributeError: property 'stamps' of 'Card' object has no setter
```

kim.stamps

✓ 꺼내는 문 (@property)

✕ 넣는 문 없음

읽기만 열어 둔 칸이에요

도장 수처럼 밖에서 바꾸면 안 되는 값은 일부러 이렇게 둬요

바꾸게 하려면 @stamps.setter나 add() 같은 메서드를 만들어요

오류처럼 보여도, 읽기만 열어 둔 것도 캡슐화예요.

6. 실습

도장 카드 ①: 열 개에서 멈추고, 읽기만 열어 두기

[1]

```
class Card:
    goal = 10

    def __init__(self, name):
        self.name = name
        self.__stamps = 0   ← 도장 칸은 밑줄 두 개로 감춰요

    def add(self):
        if self.__stamps < Card.goal:   ← 문 안에서 열 개를 넘지 않게 검사
            self.__stamps = self.__stamps + 1

    @property   ← 읽기만: setter는 일부러 안 만들어요
    def stamps(self):
        return self.__stamps

    def __str__(self):
        return f"{self.name}: 도장 {self.__stamps}/{Card.goal}"
```

출력 없이 틀만 만드는 셀이에요

6. 실습

도장 카드 ②: 열두 번 찍어도 열 개

[2]
```
kim = Card("kim")
for i in range(12):
    kim.add()          ← add를 열두 번
print(kim)
print(kim.stamps)
```

```
kim: 도장 10/10    ← __str__
10                 ← property로 꺼낸 값
```

kim.stamps

10

11번째, 12번째 add는 if에서 막혀요

밖의 코드는 add()와 stamps만 알면 돼요. 칸이 어떻게 생겼는지는 몰라도 돼요.

6. 실습

키오스크 비밀번호 ①: 옛 번호를 알아야 바꿔요

[3]

```
class Kiosk:
    def __init__(self, password):
        self.__password = password  ← 재료로 받은 번호를 감춰요

    def login(self, pw):
        if pw == self.__password:
            print("관리자 화면을 열어요")    ← 맞는지만 확인해요
        else:
            print("비밀번호가 틀렸어요")

    def change(self, old, new):
        if old == self.__password:
            self.__password = new
            print("비밀번호를 바꿨어요")
        else:
            print("지금 비밀번호가 틀려서 못 바꿔요")
```

← 옛 번호가 맞을 때만 바꿔요

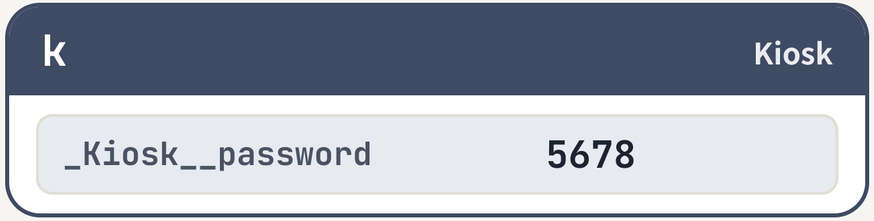
이 셀도 출력이 없어요

6. 실습

키오스크 비밀번호 ②: 틀리면 안 열리고 안 바뀌어요

[4]
```
k = Kiosk(1234)
k.login(1111)
k.change(1111, 5678)
k.change(1234, 5678)
k.login(5678)
```

비밀번호가 틀렸어요	✕ 로그인
지금 비밀번호가 틀려서 못 바꿔요	✕ 변경
비밀번호를 바꿨어요	✓ 변경
관리자 화면을 열어요	✓ 로그인



칸이 바뀐 건 옛 번호를 맞힌 한 번뿐이에요

밖의 코드는 비밀번호를 한 번도 꺼내 보지 않았어요.

정리

막히면 오류의 마지막 줄을 봐요

1	`AttributeError: 'Secret' object has no attribute '__password'` 밑줄 두 개짜리 칸을 밖에서 이름으로 찾았어요	`getter, property`
2	`AttributeError: ... no attribute '_VipCard__stamps'` 자식 틀에서 부모의 __칸을 썼어요	**밑줄 하나 _stamps**
3	`TypeError: 'int' object is not callable` property에 괄호를 붙였어요	`kim.stamps`
4	`AttributeError: property 'stamps' of 'Card' object has no setter` 읽기만 연 property에 넣었어요	**@stamps.setter, 메서드**
5	**밖에서 __password = 0 했는데 안 바뀌어요** 밖에서는 이름이 안 바뀌어서 새 칸이 생겼어요	**setter로 바꾸기**
6	**_stamps를 밖에서 바꿨더니 값이 이상해요** 약속을 어기고 검사를 건너뛰었어요	**add() 같은 메서드로**

헷갈리면 __dict__로 실제 저장된 이름을 들여다봐요.

한 장 요약

`self._stamps`

안에서만 쓰자는 약속

`self.__password`

이름이 _Secret__password로 바뀌어요

`mypass._Secret__password`

알면 우회할 수 있어요. 쓰지는 않아요

`__dict__`

저장된 이름 들여다보기

`get_password(), set_password()`

꺼내는 문, 검사하고 넣는 문

`@property`

괄호 없이 칸처럼 꺼내기

`@password.setter`

= 으로 넣어도 검사하기

`setter 없는 property`

읽기만 열어 두기

칸은 안에 두고, 밖에는 문만 열어 둬요. 파이썬의 밑줄은 자물쇠가 아니라 약속이에요.

24부터 27까지 붕어빵 틀과 도장 카드로 클래스를 만들고, 꾸미고, 물려받고, 감췄어요.